MERN Backend – Microservice-Ready Monolithic Structure

```
src/
│
├── modules/                          Feature-Based Modules
│   │
│   ├── identity/                     (Auth Module)
│   │   ├── controllers/
│   │   │     └── auth.controller.js
│   │   │
│   │   ├── services/
│   │   │     └── auth.service.js
│   │   │
│   │   ├── models/
│   │   │     └── User.js
│   │   │
│   │   ├── routes/
│   │   │     └── auth.router.js
│   │
│   ├── governance/                   (Platform Policy Module)
│   │   └── PlatformPolicy.js
│   │
│   ├── seller/                       (Seller Module)
│   │   └── product-related features
│   │
│   └── customer/                     (Customer Module)
│       └── order/cart features
│
├── routes/                           Route Aggregation Layer
│   └── server.js                      Combines all module routes
│
├── config/                           Configuration Layer
│   └── db.js                         Database connection
│
├── app.js                            Express App Configuration
│   └── middleware + route mounting
│
├── server.js                         Entry Point
│   └── DB connect + server start
│
└── .env                              Environment Variables
```

Request Flow (End-to-End)

```
Client Request
     ↓
routes/server.js
     ↓
Module Router (auth.router.js)
     ↓
Controller (auth.controller.js)
     ↓
Service (auth.service.js)
     ↓
Model (User.js)
     ↓
Database (via config/db.js)
     ↓
Response
```

Example Flow: /api/auth/register

```
POST /api/auth/register
        ↓
auth.router.js
        ↓
auth.controller.js
        ↓
auth.service.js
        ↓
User.js (DB operation)
        ↓
Response
```

Layer Responsibilities

Layer	Responsibility
Router	Maps URL to controller
Controller	Handles request and response
Service	Contains business logic
Model	Defines database structure and queries
Config	Handles system-level setup (DB, env)

Why config/db.js?

Database connection is not business logic. It is part of system configuration.

Placing it inside config/ ensures:

- Clear separation of concerns
- Reusability across modules
- Easy environment switching (development, production)
- Cleaner and scalable architecture

Key Architectural Insight

This structure is:

- Monolithic in deployment
- Microservice-ready in design

Each module behaves like an independent service:

- identity → auth service
- seller → product service
- customer → order service
- governance → policy service

Final Summary

- Modules contain business logic
- Config contains system setup
- Controllers are thin
- Services handle logic
- Structure enables easy transition to microservices